

Тема 3.2

Инкапсуляция.

Наследование.

Полиморфизм.

Абстракция

Наследование

Наследование – одна из ключевых идей ООП. Можно создать класс и унаследовать все атрибуты и методы *родительского класса*. Родительский класс должен быть более обобщённой, абстрактной версией дочернего класса.

Расширение класса через наследование

```
class Person:
```

```
    def __init__(self,n):  
        self.name=n
```

```
    def write(self):  
        print(self.name)
```

```
class Student(Person):
```

```
    def __init__(self,gr,n):  
        Person.__init__(self,n)  
        self.group=gr
```

```
    def write(self):  
        print(self.name,self.group)
```

Ввод и вывод:

```
>>> p=Person("Petya")
```

```
>>> p.write()
```

```
Petya
```

```
>>> s=Student(23,"Vasya")
```

```
>>> s.write()
```

```
Vasya 23
```

```
>>>
```

Полиморфизм

Полиморфизм – разное поведение одного и того же метода в разных классах, при этом действия, совершаемые с объектами, могут существенно различаться.

```
class T1:
    n=10
    def total(self,N):
        self.total = int(self.n) + int(N)
class T2:
    def total(self,s):
        self.total = len(str(s))

t1 = T1()
t2 = T2()
t1.total(45)
t2.total(45)
print (t1.total)      # Вывод: 55
print (t2.total)      # Вывод: 2
```

Пример программы «Банк»

Реализовать работу банка (сотрудник, клиент, вклад, кредит). В программе должны быть классы и объекты, принадлежащие разным классам; один объект с помощью метода своего класса должен так или иначе изменять данные другого объекта.

Банк

Расширение класса через наследование

Использование методов

```
class Person():                                # родительский класс
    def __init__(self, fio):                    # метод-конструктор
        self.fio = fio
        #print("Person", self.fio, "created")

    def __str__(self):                           # возвращает строку, которая
        return self.fio                        # содержит значение атрибута fio
```

Продолжение следует

Пример программы «Банк»

Банк (продолжение)

```
class Sotrudnik(Person):           # дочерний класс
    def __init__(self,fio,job_title): # унаследовал атрибут
        Person.__init__(self,fio)    # родительского класса fio
        self.job_title = job_title
        print(self.job_title,"-",self.fio,"\n")

    def kredit(self,client):
        client.dolg()
        print(self.fio,"оформил кредит \n")

    def vklad(self,client):
        client.dohod()
        print(self.fio,"оформил вклад \n")

# Продолжение следует
```

Пример программы «Банк»

Банк (продолжение)

```
class Client(Person):
```

```
    def __init__(self, fio, sum_vklada, sum_kr):
```

```
        Person.__init__(self, fio)
```

```
        self.sum_vklada = sum_vklada
```

```
        self.sum_kr = sum_kr
```

```
        print("Клиент:", self.fio, "Сумма кредита:", self.sum_kr,  
              «Сумма вклада:", self.sum_vklada, "\n")
```

Продолжение следует

Пример программы «Банк»

Банк (продолжение)

```
def dohod(self,persent=0.03):  
    self.sum_vklada = float(self.sum_vklada*(1+persent))  
  
def dolg(self,persent=0.18,amount=0):  
    self.sum_kr = float(self.sum_kr*(1+persent))  
    self.sum_kr -= amount  
  
def out(self):  
    print("Клиент:", self.fio,"Сумма долга:", self.sum_kr)  
    print("Клиент:", self.fio,"Доход:", self.sum_vklada)
```

Продолжение следует

Пример программы «Банк»

Банк (продолжение)

```
name_s = input("Введите ФИО сотрудника:")
```

```
job_s = input("Введите должность:")
```

```
name_c = input("Введите ФИО клиента:")
```

```
svk = int(input("Введите сумму вклада:"))
```

```
skr = int(input("Введите сумму кредита:"))
```

```
obj1 = Sotrudnik(name_s,job_s)
```

```
obj2 = Client(name_c,svk,skr)
```

```
obj1.vklad(obj2)
```

```
obj1.kredit(obj2)
```

```
obj2.out()
```

```
input("\n\nНажмите Enter чтобы выйти.")
```


Пример программы «Банк»

Вывод:

Введите ФИО сотрудника:Иванов

Введите должность:операционист

Введите ФИО клиента:Петров

Введите сумму вклада:100

Введите сумму кредита:1000

операционист - Иванов

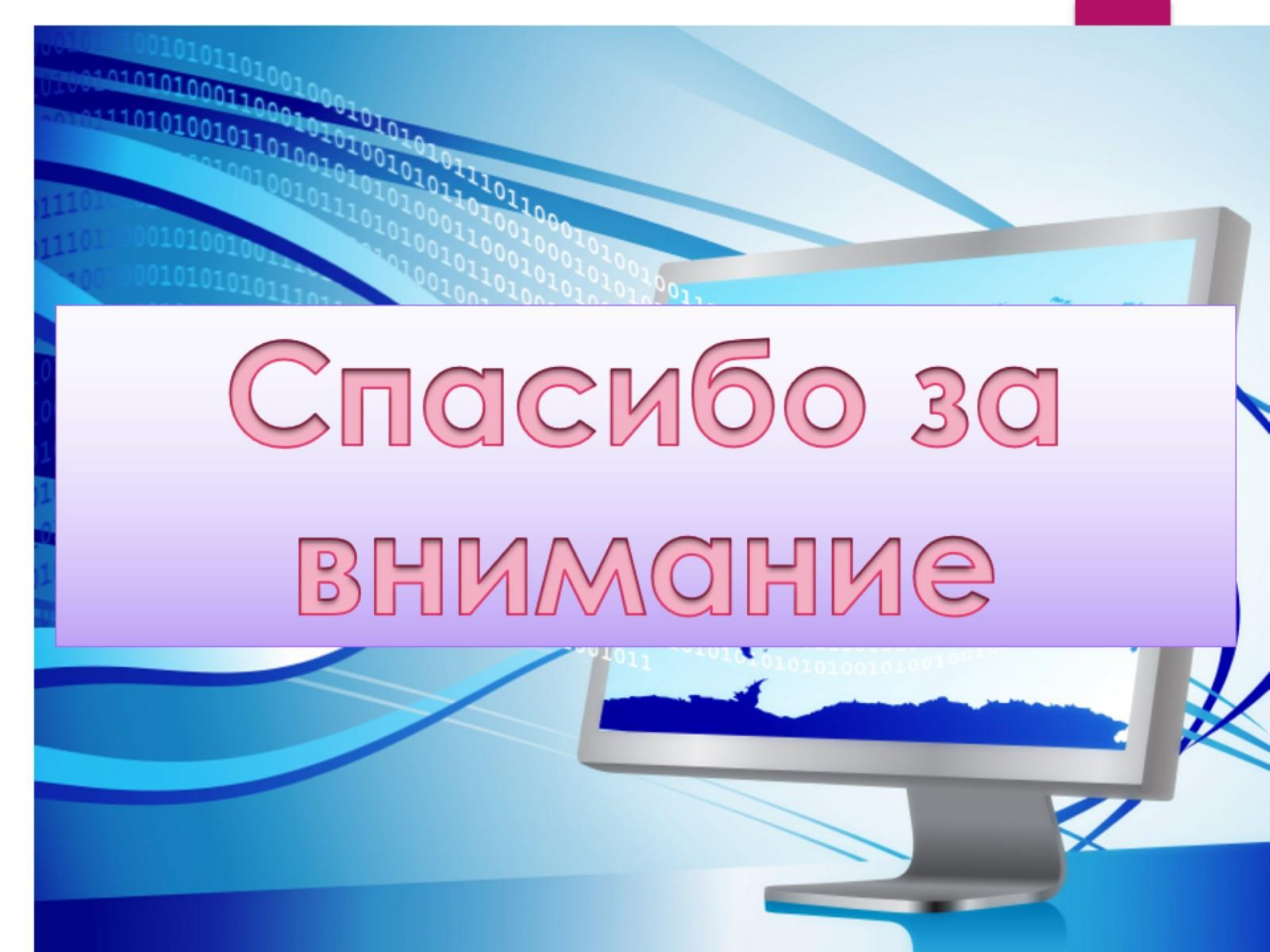
Клиент: Петров Сумма кредита: 1000 Сумма вклада: 100

Иванов оформил вклад

Иванов оформил кредит

Клиент: Петров Сумма долга: 1180.0

Клиент: Петров Доход: 103.0

The background features a blue gradient with stylized, flowing lines. Binary code (0s and 1s) is scattered across the scene, appearing to flow from the top left towards the right. Two computer monitors are visible: one in the upper right and another in the lower right. The lower monitor displays a dark, silhouetted landscape against a lighter blue sky. A large, semi-transparent rectangular box with a light purple-to-white gradient is centered over the image, containing the text.

Спасибо за
внимание